

Evaluating Classifiers and Regressors: Splits, Metrics, Baselines (Lab 2)

CS 53744 Machine Learning Project

Due Date : 11:59 PM, September 23, 2026

Instructor: Prof. Jongmin Lee

How to use this note:

- Read it before the lab session. The lab will not repeat this material.
- Covered: holdout splits, the confusion matrix and four metrics, the majority-class baseline, data leakage, controlled comparison, and the regression metrics (MAE, RMSE, R^2) with the predict-the-mean baseline. The emphasis is on what each method assumes and where it breaks.
- Not covered: code and function calls. See the docstrings in the skeleton `src/lab02.py`.
- Rules for this lab (environment, tasks, submission) are in the `README.md` in this folder. Course-wide policy — grading and AI use — was given in the policy slides of the first class and is not repeated here.

1 Learning objectives

1. Explain why evaluation needs held-out data, and what stratification, a fixed seed, and leakage prevention each protect against. (Task 1)
2. Derive accuracy, precision, recall, and F1 from the four cells of a confusion matrix, including the division-by-zero cases. (Task 2)
3. Build the majority-class baseline for a dataset and use it to calibrate what you report. (Task 3)
4. Explain why preprocessing must learn its parameters from the training data only. (Task 3)
5. Design a controlled experiment in which one factor varies and everything else is fixed. (Task 4)
6. Derive MAE, RMSE, and R^2 for a continuous target, and calibrate regression numbers with the predict-the-mean baseline. (Tasks 5, 6)

2 Background: how do you know the model is any good?

Lab 1 cleaned a table. Now you build a model from that data and decide whether it is worth anything. That decision is harder than it looks.

A trained model is making a claim about the future: that it can predict no-shows this well on appointments it has never seen. You cannot check that claim by testing on data the model has already memorized, because it is an exam with the answers attached. So you set part of the data aside, keep it out of training, and grade only on that part at the end.

Setting data aside is not enough on its own. In the Lab 2 dataset, 65% of appointments are kept and 35% are no-shows. On data like this you can score 65% accuracy by learning

nothing at all and predicting “kept” every time. When your model reports 69%, there is no way to say whether that is good until the 65% sits next to it.

There is one more problem. Information from the held-out data leaks into training very easily. It happens if you normalize the whole dataset once before splitting it, because the mean and standard deviation you used already contain the validation rows. Mistakes like this raise no error and only raise the score. The gap shows up when the model is actually used.

What you build in Lab 2 is less a model than the evaluation procedure that enforces these three things. Grading follows the same logic: not how high the accuracy is, but whether the split is correct, the metrics are computed correctly, and the comparison is honest. You then run the same procedure once more with a continuous target, predicting waiting times at the same clinic: the metrics change to MAE, RMSE, and R^2 , and the discipline stays the same. Every later lab and the team project reuse this procedure.

The six steps of Lab 2:

- **Split:** stratify and fix the seed. Nothing is learned before this point. (Task 1)
- **Metrics:** compute the four metrics from the confusion matrix yourself, with numpy only. (Task 2)
- **Baseline and pipeline:** set the reference point, and bind preprocessing to the model so leakage cannot happen. (Task 3)
- **Controlled comparison:** vary only the model and hold everything else fixed. (Task 4)
- **Regression metrics:** compute MAE, RMSE, and R^2 from the residuals yourself, with numpy only. (Task 5)
- **Regression comparison:** predict-the-mean baseline, linear regression, and k -nearest neighbours under identical conditions. (Task 6)

3 Key concepts

3.1 Setting data aside

Twenty percent of the data is kept out of training, and performance is measured only on that hidden part. Three conditions make the measurement trustworthy (Figure 1).

- **Stratification:** split so that both sides keep the original class ratio. Split a 35% positive dataset at random and the validation set can come out at 30% or 40%, which moves the metrics for reasons that have nothing to do with the model.
- **Fixed seed:** fix the shuffling so the split is reproducible. The same seed must give the same split, and a different seed must give a different one. The hidden tests check both.
- **No leakage:** learn nothing from the data before splitting. Scaling constants, imputation statistics, feature selection, and models are all included.

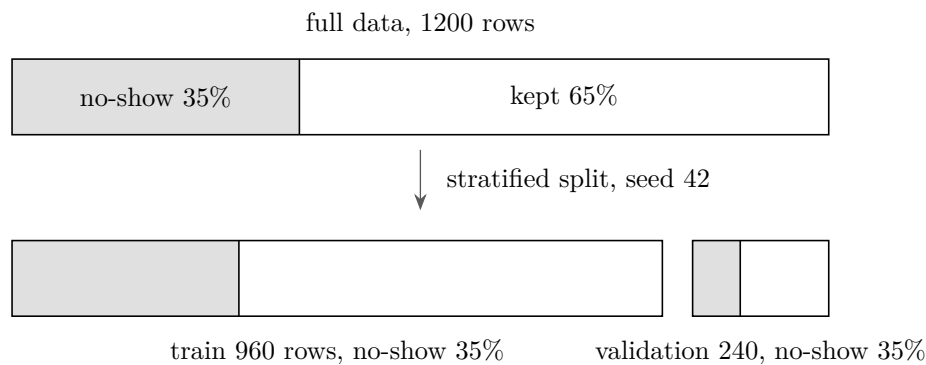


Figure 1. A stratified holdout split. The data is cut into 80% train and 20% validation, and both sides keep the original class ratio. Widths are proportional to row counts.

Assumptions and limits:

- Rows are assumed independent. For time series, or data with several records per person, a random split is itself leakage: you train on Monday’s record and test on the same patient’s Tuesday record. Split such data by time or by person instead.
- One split is one measurement. Change the seed and the numbers move a little.

3.2 The confusion matrix and four metrics

In binary classification every prediction lands in one of four cells (Figure 2): two ways to be right and two ways to be wrong.

	predicted 1	predicted 0	
actual 1 (no-show)	TP	FN (missed)	} recall divides by TP + FN
actual 0 (kept)	FP (false alarm)	TN	
	} precision divides by TP + FP		

Figure 2. The four cells of the confusion matrix. Precision divides down a column, recall across a row. Because the denominators differ, moving the decision threshold moves the two in opposite directions.

The four metrics:

- **Accuracy:** $(TP + TN)/N$. The share of all predictions that are right.
- **Precision:** $TP/(TP + FP)$. Of everything called positive, the share that is.
- **Recall:** $TP/(TP + FN)$. Of all actual positives, the share found.
- **F1:** the harmonic mean of precision and recall. Raising one alone does not raise it.

Which metric to watch:

- When false positives are expensive, watch precision. A spam filter that deletes real mail is the standard example.

- When false negatives are expensive, watch recall. Disease screening is the standard example, and appointment no-shows partly belong here: a no-show you fail to predict wastes a slot.

Division by zero:

If the model never predicts the positive class, $TP + FP = 0$ and precision becomes $0/0$. The spec defines such values as 0.

Spec (Task 2)

$\text{precision} = TP / (TP + FP)$ (define as 0.0 if $TP + FP == 0$)

$\text{recall} = TP / (TP + FN)$ (define as 0.0 if $TP + FN == 0$)

$f1 = 2 * P * R / (P + R)$ (define as 0.0 if $P + R == 0$)

Return plain Python floats rounded to 4 decimals.

This is not a formality. The baseline described next is exactly this case, and without the convention its precision and F1 would be undefined rather than 0, so it could not go in the same table as the other models.

3.3 The majority-class baseline

The majority-class baseline predicts the most common class for every input. It learns nothing, yet on imbalanced data its accuracy equals the majority share. Figure 3 is the comparison on this dataset.

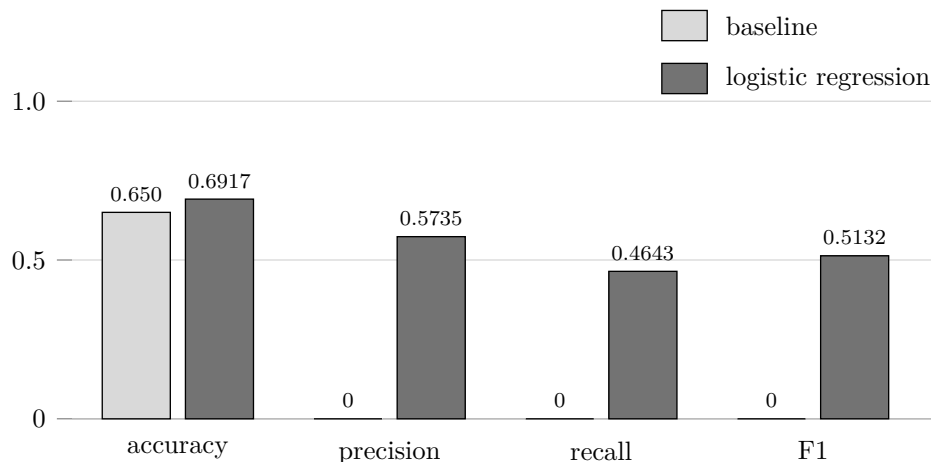


Figure 3. Baseline against logistic regression. Accuracy differs by 4 points, while the baseline starts from zero on the other three metrics. Whether the model actually finds no-shows shows up there.

How to read it:

- A model whose accuracy sits near the baseline has learned nothing, whatever the absolute number looks like.
- A metric that is already high for the baseline is no evidence of learning. Accuracy on imbalanced data is exactly such a metric.
- So the honest description of this result is not “69% accuracy” but “4 points of accuracy above the baseline, and F1 of 0.51 against 0.0, so the model does find no-shows”.
- Every results table in this course gets a baseline row.

3.4 Preprocessing learns from the training data only

Features come on different scales. Days of lead time run from 0 to 60 while an SMS-reminder flag is 0 or 1. Distance-based and gradient-based models are distorted by that difference, so features are standardized: centered and scaled using a mean and a standard deviation computed from data.

The catch is that this mean and standard deviation are learned from data exactly like model weights. Compute them on the full dataset and the validation rows have helped build the transform that will later be used to evaluate them. No labels were touched, and it is still leakage. Figure 4 shows the wrong order and the right one.

The wrong way



The right way

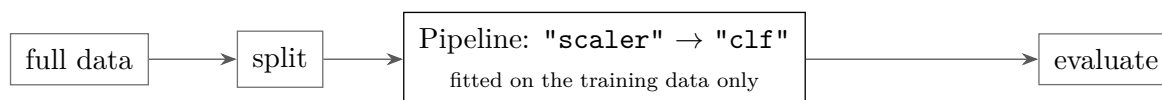


Figure 4. The order that leaks and the order that does not. Standardizing before the split puts the validation rows into the transform. Binding the steps into a pipeline keeps preprocessing on the training data only.

What to know:

- The principle is not limited to scaling. Every step that learns something is covered: imputation statistics, vocabularies, feature selection, class weights.
- In practice you prevent the mistake structurally, by binding preprocessing and model into one object fitted on the training data. That is what scikit-learn's Pipeline is for.
- The two pipeline steps must be named exactly "scaler" and "clf". The hidden tests inspect the structure by those names.

3.5 Controlled comparison: change one thing at a time

Task 4 compares three things on the same data: the majority-class baseline, logistic regression, and k -nearest neighbours with $k = 15$.

- To claim one is better than another, the comparison has to be controlled: same split, same seed, same preprocessing, same metric definitions, and only the model changes.
- Change the model and re-split at the same time and there is no way to know which change produced the difference.
- The ablation study in Lab 8 and the team-project experiments use the same design with more factors.
- One split is one measurement, not the truth. Writing that limitation into your report scores better than hiding it.

3.6 A continuous target: regression metrics and the mean baseline

The data for Tasks 5 and 6 is 900 visits to the same clinic, and the target is the waiting time in minutes rather than a no-show flag. With a continuous target there is no confusion matrix, so the metrics are built from the residuals $e_i = y_i - \hat{y}_i$.

The three metrics:

- **MAE:** $\frac{1}{n} \sum |e_i|$. How many minutes off on average, in the target's own units.
- **RMSE:** $\sqrt{\frac{1}{n} \sum e_i^2}$. Squares the errors before averaging, so it is sensitive to large misses. For any predictions $\text{RMSE} \geq \text{MAE}$, and a wide gap between them means a few large errors are hiding in the average.
- **R^2 :** $1 - \sum e_i^2 / \sum (y_i - \bar{y})^2$. How much of the variance the model explains, measured against predicting the mean.

The mean baseline:

- The regression counterpart of the majority-class baseline predicts the training-set mean waiting time for every visit.
- That comparison is exactly the denominator of R^2 . On validation data the mean baseline's R^2 lands near 0, and not exactly at 0, because the mean it predicts comes from the training rows. The reference solution gets -0.0056 .
- R^2 can go negative. That means worse than predicting the mean, and a hidden test checks the case.

Spec (Task 5)

```
mae = mean(|e_i|), rmse = sqrt(mean(e_i ** 2))
r2 = 1 - sum(e_i ** 2) / sum((y_i - ybar) ** 2) (define as 0.0 if the de-
nominator is 0)
Return plain Python floats rounded to 4 decimals.
```

Assumptions and limits:

- MAE and RMSE inherit the target's units, so they cannot be compared across datasets. When that comparison is needed, R^2 is the one to look at.
- A high R^2 does not mean the predictions are good enough for the clinic to use. It is a relative claim against the mean, so report the MAE in minutes next to it.
- The split in Task 6 is not stratified: a continuous target has no classes to stratify on, so `split_data` is not reused and `train_test_split` runs without `stratify`. Everything else (the fixed seed, splitting before learning, the pipeline) carries over from classification unchanged.

4 Worked example: metrics on eight appointments

Work through the eight appointments in Table 1, where 1 means no-show.

Table 1. The eight appointments of the worked example. Actual labels and predictions side by side. The four metrics and the majority-class baseline are computed by hand from these eight rows.

appointment	1	2	3	4	5	6	7	8
actual	1	0	1	1	0	0	0	0
predicted	1	0	0	1	0	1	1	0

Counting the four cells:

- TP 2 (#1, #4), FN 1 (#3), FP 2 (#6, #7), TN 3 (#2, #5, #8)

Computing the metrics:

1. Accuracy = $(2 + 3)/8 = \mathbf{0.625}$
2. Precision = $2/(2 + 2) = \mathbf{0.500}$
3. Recall = $2/(2 + 1) \approx \mathbf{0.667}$
4. F1 = $2 \cdot (0.5 \cdot 0.667)/(0.5 + 0.667) \approx \mathbf{0.571}$

The baseline on the same eight:

- The actual labels are five 0s and three 1s, so the majority class is 0 and the baseline predicts 0 everywhere.
- Accuracy = $5/8 = \mathbf{0.625}$, identical to the model above.
- Precision, recall, and F1 are all **0**. It never predicted a positive.

Same accuracy, entirely different usefulness: the model found two of the three no-shows and the baseline found none. This small example is the whole reason to report four metrics next to a baseline.

Regression on four visits (Task 5):

- Suppose four visits actually waited 30, 50, 60, and 20 minutes and a model predicted 35, 45, 55, and 25. The residuals are $-5, +5, +5, -5$.
- 1. MAE = $(5 + 5 + 5 + 5)/4 = \mathbf{5.0}$ minutes
- 2. RMSE = $\sqrt{(25 + 25 + 25 + 25)/4} = \mathbf{5.0}$ minutes, equal to the MAE because all four errors have the same size.
- 3. Mean $\bar{y} = 40$, denominator $\sum (y_i - \bar{y})^2 = 100 + 100 + 400 + 400 = 1000$, numerator $\sum e_i^2 = 100$, so $R^2 = 1 - 100/1000 = \mathbf{0.9}$
- Predict the mean of 40 minutes everywhere instead and MAE = 15 minutes with $R^2 = 1 - 1000/1000 = \mathbf{0}$. The arithmetic makes the identity plain: $R^2 = 0$ is the mean baseline.

5 Common mistakes

1. **Learning something before the split.** The validation score comes out slightly better and the hidden leakage check catches it. Split first, and bind preprocessing to the model so it only ever sees training data.

2. **Forgetting to stratify.** On a small validation set the class ratio wobbles, and the metrics jump between runs for reasons unrelated to the model.
3. **Undefined metrics at the edges.** When the model never predicts one class you get a divide-by-zero warning or NaN. Put the spec's zero convention into your code explicitly.
4. **Re-splitting for each model.** Reordering your code then changes the conclusion. Split once and use that split for every model.
5. **Reading accuracy without a baseline.** This is the most common way a classification report overrates itself. Adding the baseline row makes the claim honest, and often changes it.
6. **Making the same mistake in regression.** Whether an MAE of 4.5 minutes is good is unknowable until the mean baseline's 28.5 minutes sits next to it. Regression tables get a baseline row too.

6 How this maps to the lab tasks

Table 2. Tasks matched to concept sections. Which section each task draws on, and what grading looks at. Worth checking once before you write the report.

Lab task	Concept	What grading looks at
Task 1. Data split	§3.1	stratification, fixed seed, leak-free holdout
Task 2. Metrics from scratch	§3.2, §4	metrics as ratios of the four cells, edge cases
Task 3. Baseline and pipeline	§3.3, §3.4	a reference point, preprocessing fitted on train only
Task 4. Model comparison	§3.5	one factor varies, everything else fixed
Task 5. Regression metrics	§3.6, §4	three metrics from residuals, zero denominator
Task 6. Regression comparison	§3.5, §3.6	mean baseline, unstratified split, same conditions
Report figures 1–3	§3.3, §3.5	class balance, feature separation, honest comparison

7 Self-check

1. A friend reports 93% accuracy on a dataset that is 93% negative. What do you ask first, and which answer would make the model worthless?
2. Why must the same seed give the same split and a different seed a different one? Which bug does each property catch?
3. A model predicts every sample as positive. State its recall without computing anything. If the positive rate is 35%, what is its precision?
4. Standardization uses no labels, yet fitting it on the full dataset is leakage. Explain why in two sentences.
5. In this clinic setting, which costs more, a false positive or a false negative? Which metric should your report weight accordingly?

6. A regression model reports $R^2 = -0.2$. What is it worse than, and what bug should you suspect first?

These questions are fair game for the oral check. You must be able to explain every line you submit.

8 Further reading

- scikit-learn User Guide, Common pitfalls: data leakage
https://scikit-learn.org/stable/common_pitfalls.html#data-leakage
- scikit-learn User Guide, Model evaluation
https://scikit-learn.org/stable/modules/model_evaluation.html
- Kapoor & Narayanan, *Leakage and the Reproducibility Crisis in ML-based Science*, 2023
<https://arxiv.org/abs/2207.07048>